

From Fleet to Lab: Revisiting the Security and Complexity of Industrial Rowhammer Mitigation

Hritvik Taneja
Georgia Institute of Technology
htaneja3@gatech.edu

Moinuddin Qureshi
Georgia Institute of Technology
moin@gatech.edu

Abstract—This paper studies efficient and secure Rowhammer mitigation at the Memory-Controller (MC). Rowhammer mitigation faces a fundamental tradeoff between tracking storage and mitigation rate: precise trackers (such as Misra-Gries) avoid unnecessary mitigations but require large CAM structures, whereas sampling-based schemes (such as PARA) require no storage but incur frequent mitigations even when not under attack. Microsoft recently deployed Sigries, an MC-side Rowhammer defense that combines an under-provisioned Misra-Gries tracker with a row-sampling fallback, in its Azure Cobalt 200 SoC. Sigries observed that the tracker-to-sampling transition can be insecure, and claimed the reverse transition is always safe. Our analysis shows that this transition is also vulnerable, and that a *Round-Robin Attack* across sub-banks reduces the Mean-Time-To-Failure of Sigries to about one second, eight orders of magnitude below the 13 years with PARA. Sigries also suffers from CAM complexity and high storage overheads. Our goal is to develop a solution that is fully secure and minimizes the storage and complexity of Sigries, while matching its performance.

Our proposal, *FiRM* (*Filtered Rowhammer Mitigation*), is based on the key insight that for a secure design the tracking-mode and sampling-mode should not be configured independently, but co-designed to ensure that the system remains secure not only during the two modes but also during the two transitions. *FiRM* also avoids the complexity of Sigries by replacing the CAM-based tracker with simple SRAM filters. As benign workloads do not exceed the filtering threshold, *FiRM* incurs no slowdown for benign workloads. To handle stressful patterns, we propose two variants. First, *FiRM-P* (*probabilistic*), which relies on PARA; however, it uses varying probabilities during transitions and steady state to ensure both security and low performance overhead. *FiRM-P* is secure, has less than half the storage overhead of Sigries, and provides similar performance. Second, *FiRM-D* (*deterministic*), which provides guaranteed deterministic security by modulating the rate of mitigation (zero for benign workloads and non-zero under attack). *FiRM-D* requires less than two-thirds the storage of Sigries, does not require any CAM, and has zero slowdown for benign workloads. Our paper shows that a principled approach can avoid both the insecurity and the complexity of Sigries.

I. INTRODUCTION

DRAM scaling has enabled denser chips but it has also made DRAM increasingly susceptible to data-disturbance errors, such as Rowhammer [13]. Rowhammer occurs when frequent activations to a DRAM row cause bit-flips in nearby rows. Rowhammer is not merely a reliability concern but a serious security threat [14], [34]. The severity of Rowhammer is characterized by the *Rowhammer Threshold* (T_{RH}), the minimum number of activations required to induce a bit-flip.

Why Memory-Controller Side Mitigation? A typical hardware Rowhammer defense has two parts: a *tracker* that identifies aggressor rows, and a *mitigation* that refreshes the victim rows. Mitigation can be done either inside the DRAM (in-DRAM) or at the Memory Controller (MC). Commercially deployed in-DRAM trackers such as TRR have been broken [4], [6], [8]. Principled in-DRAM solutions such as *Per-Row Activation Counting* (PRAC) incur significant slowdown (about 8.4%) even when the system is not under an attack. MC-based mitigations can provide strong security while incurring lower overheads. Furthermore, recent trends in hyperscalers [5] to reuse DIMMs from decommissioned servers mean that currently deployed DDR5 DIMMs (insecure against Rowhammer) may remain in use for more than a decade, further boosting the need for MC-side Rowhammer defense.

The Storage vs. Mitigation Tradeoff: MC-side trackers fall into two classes. *Deterministic* (counter-based) trackers, such as Graphene [23], use the Misra-Gries algorithm to identify the top- K most activated rows. As these trackers issue mitigations only when needed, benign workloads experience virtually zero mitigations. However, they require a CAM structure with hundreds of entries (per bank), making them impractical for commercial adoption. At the other end, *probabilistic* (sampling-based) trackers, such as PARA [13], select a row for mitigation with probability p . While they do not need any storage, they issue frequent mitigations even when the system is not under an attack. Ideally, we want the mitigation-efficiency of counter-based trackers and storage-efficiency of sampling-based trackers, as shown in Figure 1(a).

Sigries: Recently, Microsoft deployed *Sigries* [32] (in Azure Cobalt 200), an MC-side Rowhammer defense that targets the above trade-off. Sigries splits the bank into several sub-banks, and equips each sub-bank with an *under-provisioned* Misra-Gries tracker (containing a couple of dozen entries) and row-sampling fallback, as shown in Figure 1(b). The tracker is sufficient to handle benign workloads without triggering mitigations, thus benign workloads incur zero slowdown. When an access pattern overflows the tracker, the design falls back to PARA, providing probabilistic protection at negligible additional storage. As the tracker is under-provisioned, each sub-bank of Sigries needs far fewer CAM entries than per-bank Graphene. However, Sigries still incurs the complexity of a CAM lookup of the sub-bank entries on each activation.

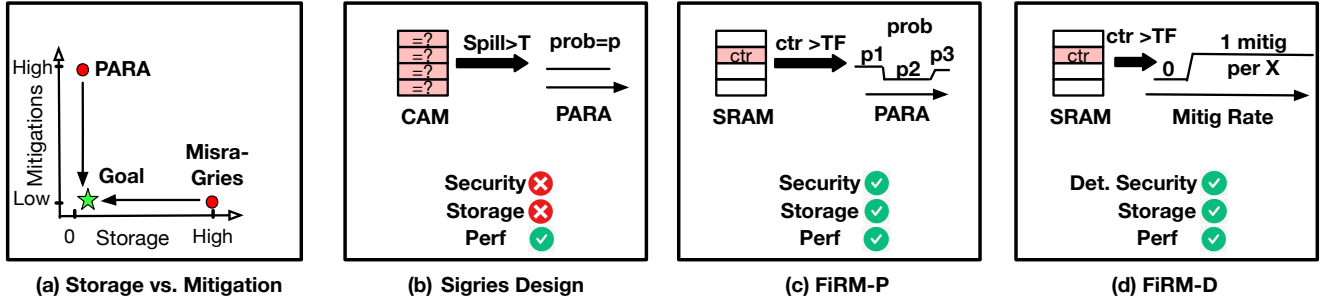


Fig. 1. (a) Counter-based trackers need more storage, whereas PARA needs more frequent mitigations; ideally, we want low storage and low mitigation. (b) Sigries uses a per-sub-bank shortened Misra-Gries tracker (CAM) and a uniform rate for PARA as fallback; however, it suffers from both insecurity and complexity. (c) Our proposal, FiRM-P, provides principled Filtered Rowhammer Mitigation and ensures security in both modes and transitions, and avoids CAM complexity by using simple SRAM filters. (d) Our proposal, FiRM-D, provides deterministic security by modulating the mitigation rate of a region.

The two components of Sigries (Misra-Gries and PARA) are each designed independently to be secure within their mode. However, combinations of two things that are independently secure can still be insecure. Sigries identifies that the transition from tracker to sampling can be unsafe and handles this by limiting the vulnerability windows to less than 1 hour per year (per sub-bank). It does so by running the sampling-mode for several minutes on each transition. Our paper investigates whether Sigries has any additional unknown vulnerabilities.

While the Sigries paper asserts that the transition from sampling-mode to tracker-mode is *always* safe, our analysis shows otherwise. Our key observation is that tracker-mode is safe only if the row started with zero unmitigated activations, however, for Sigries, the row would have hundreds of unmitigated activations (due to prior sampling-mode), making the transition unsafe. We also show that while the target vulnerability of one hour per year (per sub-bank) may seem low, an attack that *Round-Robins* multiple sub-banks can cause **a failure within a second**, reducing the system MTTF by eight orders of magnitude. The **goal** of this paper is to design a secure mitigation that also avoids the complexity of Sigries.

Insight and Solution. The root cause of the vulnerability of Sigries is that the two solutions (tracker and sampling) are independently designed to be secure in isolation, assuming the given row starts with zero activations. When the two schemes are combined, this assumption gets violated, which causes unsafe behavior. The key insight of our paper is that to ensure security both modes should be co-designed to ensure that not only are they secure in isolation but also during the transition from one to another. Our proposal, **FiRM (Filtered Rowhammer Mitigation)**, is built on this insight. FiRM co-designs the filter and the fallback so that the activation budget of an attacker is bounded across all modes and transitions.

We also observe that the filter does not need the precision of Misra-Gries, it simply needs to track if the benign traffic to the given region is below a certain threshold. FiRM therefore replaces the CAM-based tracker of Sigries with a simple *SRAM filter*: an untagged, direct-mapped table of counters that requires no associative lookup. As benign workloads do not exceed the *Filtering Threshold* (T_F), FiRM incurs *zero* slowdown on benign workloads, while relying on the fallback under attacks. We propose two variants that differ in fallback.

Our first design, **FiRM-P (Probabilistic)**, uses PARA as its fallback, however, unlike Sigries, it does not use a uniform probability for mitigation. The activations an attacker can accrue while crossing a mode boundary are larger than those accrued in steady state, so FiRM-P uses an elevated probability during the transition windows and relaxes to a lower steady-state probability during steady-state, as shown in Figure 1(c). This lets FiRM-P avoid security vulnerabilities during transitions while still ensuring similar steady-state performance to Sigries during sampling mode. FiRM-P is secure by construction, avoids CAM complexity, requires less than half the storage, and has similar performance as Sigries.

Our second design, **FiRM-D (Deterministic)**, avoids reliance on random number generator (RNG), which can be a source of both design complexity and insecurity [25]. Rather than sampling, FiRM-D modulates the *rate* of mitigation of a region based on the activation count. As shown in Figure 1(d), the rate is zero when the filter is below T_F (benign workloads) and non-zero thereafter to ensure security for the attacked row. FiRM-D provides strong deterministic security, uses no CAM or RNG, requires less than two-thirds the storage of Sigries, and has zero slowdown for benign workloads.

Contributions: Our paper makes the following contributions:

- 1) We show that Sigries has a previously unidentified vulnerability during the heavy-to-lite transition. We also show that under multi-sub-bank attack the MTTF of Sigries reduces to 1 second, instead of 13 years for PARA.
- 2) We propose *FiRM*, a principled filtered mitigation, that configures the tracking and sampling components jointly, so that security holds across both modes and transitions.
- 3) We show that FiRM has lower storage and complexity than Sigries, as it relies on simple SRAM filters instead of Misra-Gries. FiRM-P has zero slowdown for benign workloads and the same as Sigries under attack.
- 4) We show that filtered Rowhammer mitigation need not rely on a probabilistic fallback. *FiRM-D* is a deterministic filtered Rowhammer mitigation that avoids reliance on RNG and has less than two-thirds the storage of Sigries.

Our paper shows that neither the security tradeoff nor the CAM complexity of Sigries is essential for designing an effective filtered Rowhammer mitigation. A principled design can deliver both strong security and simplified implementation.

II. BACKGROUND AND MOTIVATION

A. Threat Model

We assume an attacker who knows the mitigation algorithm and all of its parameters (tracker size, thresholds, sampling probability, and the mode). The only secret is the output of the random number generator (RNG) used by a probabilistic mitigation. A design is *secure* if it can guarantee that the number of activations between consecutive mitigation/refresh remains no more than T_{RHD} . RowPress [16] is not considered, as it can be handled with page-closure policy [33].

B. DRAM Architecture and Parameters

Accessing DRAM requires an ACT to move a row into the row-buffer, and two ACTs to the same bank must be separated by the row-cycle time t_{RC} . To retain data, every row must be refreshed within a refresh window t_{REFW} ; the MC issues a refresh every t_{REFI} , which occupies the bank for t_{RFC} . Table I shows the DDR5 parameters. An attacker can do up to 620K activations to bank within the refresh window t_{REFW} .

TABLE I
DDR5 PARAMETERS (DDR5-6000AN, 32-GIGABIT CHIP) [20]

Parameter	Description	Value
t_{REFW}	Refresh window	32 ms
t_{REFI}	Interval between REF commands	3900 ns
t_{RC}	Time between successive ACTs to a bank	46 ns
t_{DRFMsb}	Same-bank DRFM latency (8 banks)	240 ns
t_{DRFMab}	All-bank DRFM latency (32 banks)	280 ns

C. Rowhammer and Thresholds

Rowhammer [13] occurs when frequent activations of an *aggressor* row cause bit-flips in nearby *victim* rows. The Rowhammer Threshold is typically quoted for single-sided (T_{RHS}) or double-sided (T_{RHD}) patterns. While thresholds dropped from 140K (T_{RHS}) in 2014 [13] to 4.8K (T_{RHD}) in 2020 [11], recent DDR5 characterization shows T_{RHD} has stayed within 5K–12K range since then [12], [19]. The Sigries paper [32] observes that *the DRAM industry does not expect thresholds to fall below a few thousand*. Like Sigries, we therefore target thresholds of a few thousand activations: our **default T_{RHD} is 3K**, with sensitivity to 2K and 4K.

D. Mitigation: DRFM and Aggressor Detection

To mitigate Rowhammer at the Memory-Controller (MC), we need to track aggressor rows and refresh the victim rows.

DRFM: The MC cannot refresh a victim row directly, as the aggressor-to-victim mapping [24], [36] is internal to the DRAM. DDR5 provides *Directed Refresh Management* (DRFM) [26], [38], which lets the MC sample the given row (using a special precharge command) to the DRAM chip and then issue the DRFM command to refresh the victims associated with the sampled row. *DRFMsb* mitigates the row of one bank per eight bank-groups and stalls those 8 banks for 240 ns, whereas *DRFMab* mitigates and stalls all 32 banks of the given rank for 280 ns. As DRFM stalls multiple banks, issuing a

DRFM to mitigate only a single bank is inefficient. Therefore, DREAM [38] recently proposed delaying the issue of DRFM until the bank needs to sample another row, thus allowing sibling banks more time to perform their own sampling (and later do concurrent mitigation under a single DRFM), resulting in reduced number of DRFM and associated stalls.

PARA: PARA [13] is a probabilistic method to identify aggressor rows. On an activation, the MC mitigates the neighbors of the activated row with probability p . PARA has a non-zero probability of failure. We target a failure rate comparable to that of naturally occurring errors, namely 1 failure per 10K years per bank [2]. For this target, we use $p = 20/T_{RHD}$, which closely matches the rate obtained by the recurrence-based method of Saroiu and Wolman [31] (see Appendix B).¹

Table II shows the selection probability (p) and the overhead of PARA for Naive-DRFM (issue DRFM after sampling) and DREAM (delay DRFM until next sampling) under a pattern that performs continuous activations. We assume DRFMsb, so, on average, we incur a delay of 240 ns after $1/p$ activations to the bank (which requires $46/p$ ns). At T_{RHD} of 3K, Naive-DRFM causes a loss of 28% (8x stalls), whereas DREAM-DRFM causes a loss of only 3.5%. So, we assume PARA is always implemented with DREAM.

TABLE II
PARA PARAMETERS AND OVERHEAD.

T_{RHD}	p	BW Overhead	
		Naive-DRFM	DREAM-DRFM
2000	1/100	41.7%	5.2%
3000	1/150	27.8%	3.5%
4000	1/200	20.9%	2.6%

Misra-Gries (MG): PARA incurs slowdown even when the system is not under an attack. The slowdown from mitigations can be reduced by using precise tracking. The most prominent tracker is the Misra-Gries [21] algorithm used in Graphene [23]. MG tracker contains a table with K tagged-entries (each with a counter) and a *spill-counter*. On activation, if the row is contained in the table, the corresponding counter is incremented. Otherwise, the spill-counter is incremented, and if it is above the minimum-count entry in the table, that entry is overwritten with the accessed row and spill-counter. A mitigation is issued if the tracked count crosses a specific internal threshold (T_{MG}). As the tracker is periodically reset (e.g. every t_{REFW}), this causes the tracker to forget the counts of the tracked row. An attacker could do $(T_{MG} - 1)$ activations each before and after reset, and evade mitigation even after $2(T_{MG} - 1)$ activations. For safe reset, T_{MG} is set to $T_{RHD}/2$.

Table III shows the T_{MG} , the number of table entries, and the size per bank for the Misra-Gries tracker as T_{RHD} is varied from 2K to 4K. We note that each entry of MG table contains a valid bit, lock bit (to lock rows that have crossed T_{MG}), a 17-bit tag, in addition to the counter. The

¹We note that for a victim row to fail under the double-sided Rowhammer attack, both aggressors must escape mitigation during T_{RHD} activations each, so this becomes equivalent to a single-sided attack with p set to $40/T_{RHS}$.

TABLE III
TRACKER THRESHOLD AND STORAGE FOR MISRA-GRIES.

T_{RHD}	T_{MG}	Entries	Entry Size	Storage/Bank
2000	1000	623	29 bits	2258 Bytes
3000	1500	416	30 bits	1560 Bytes
4000	2000	312	30 bits	1170 Bytes

problem with MG tracking is the large number of entries per bank (416 for our default T_{RHD} of 3K), which must be searched associatively on *every* activation. The CAM (content-addressable memory) structure required to design the MG table results in significant area, complexity, and power consumption. This is the main reason unrestricted MG tracking has not been adopted commercially, even though it is a principled solution that issues only a minimal number of mitigations.

E. Sigries: Rowhammer Mitigation in Production

PARA suffers from frequent mitigations, whereas MG suffers from the complexity of large CAMs. A recent paper [32] outlines six requirements for an MC-based Rowhammer mitigation to be viable for commercial adoption:

Requirements: (R1) Minimal bandwidth and latency overheads when not under attack, (R2) No performance outliers, (R3) Liveness under attack, no microsecond-scale delays, (R4) Low hardware cost, (R5) Flexibility to handle different hardware via simple configuration changes, and (R6) Configurable security guarantees that bound the exposure to a desired level.

Design: Sigries [32], deployed by Microsoft in Azure Cobalt 200, is an MC-side defense, developed against these six requirements. Figure 2 shows the overview of Sigries. Sigries divides each bank into S sub-banks, each equipped with an under-provisioned MG tracker (TinyMG) containing only a couple dozen or so entries. TinyMG is secure as long as the spill-counter remains below T_{MG} . We call this *lite-mode* as it incurs minimal mitigation overheads. However, when the spill-counter reaches T_{MG} , Sigries transitions to a mode where each activation gets mitigated with probability p . This mode is called *heavy-mode* as it can incur frequent mitigation. Sigries remains in heavy-mode for an Epoch of at least E windows. While in heavy-mode, Sigries uses *shadow counters* to track the intensity of the attack on the sub-bank, and extends the epoch beyond E if the attack appears to be ongoing.

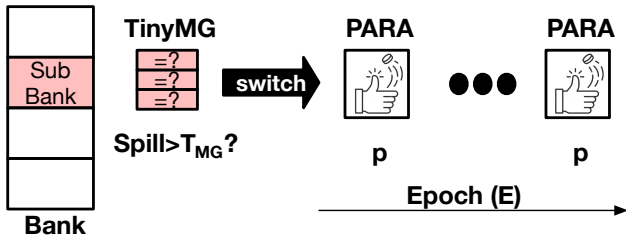


Fig. 2. Overview of Sigries: It divides the bank into several sub-banks, each provisioned with TinyMG. If the spill-counter of TinyMG exceeds T_{MG} , it switches to PARA and remains there for at least E windows, exiting only if its shadow counters indicate the attack has stopped.

Each mode is designed to be secure *in isolation*, assuming a row enters the mode with zero unmitigated activations. Therefore, T_{MG} is set to $T_{RHD}/2$ and the p is set to $20/T_{RHD}$.

Security: Sigries recognizes that the lite-mode to heavy-mode transition can cause insecurity, as a row may start with T_{MG} activations already consumed, which may not leave enough activations for PARA to operate securely. Sigries treats this as a bounded *vulnerability window* of one tREFW and amortizes it by using the heavy-mode for at least E windows after each transition (Epoch length gets extended under continued attacks). Thus, the vulnerability can be bounded to at most $1/E$. Sigries picks E such that the vulnerability is bounded per sub-bank (granularity was confirmed to us by its authors) to be within 1 hour per year ($E \geq 8760$ windows).

Performance: The key advantage of Sigries is that it provides zero slowdown for benign applications, and the mitigation overhead remains small even under attack.

Storage and Complexity: As the Sigries paper does not disclose the parameters of the deployed design, all Sigries parameters used in this paper (including T_{MG} , p , E , and the tracker size) are **estimated or inferred** from its design description for our target T_{RHD} of 3K. Without loss of generality, we assume that each TinyMG is configured to use 32 entries (which is compatible with *couple/few-dozen entries* mentioned in the Sigries paper). We then vary the number of sub-banks and identify those for which there is never a switch from lite-mode to heavy-mode, which for our system means $S \geq 8$. Table IV shows the number of entries in Graphene and Sigries (with 8 and 16 sub-banks). We use Sigries with 8 sub-banks and equip each sub-bank with a 32-entry TinyMG.

TABLE IV
NUMBER OF MG ENTRIES IN GRAPHENE AND SIGRIES (T_{RHD} OF 3K).

Design	Sub-banks	Entries/Sub-bank	Total Entries/Bank
Graphene	1	416	416
Sigries	8 (default)	32	256
Sigries	16	32	512

F. Goal of This Paper

Sigries is the first production-level Rowhammer defense described in detail in a research paper, which allows the scientific community to review the industrial design. To that end, the goal of our paper is to answer the following questions:

- (1) Does the Sigries design have any additional unknown vulnerabilities? This is important as Sigries is not a theoretical design but something deployed in commercial systems.
- (2) What is the impact of Sigries vulnerability target (of one hour per year) on the Mean-Time-To-Failure of the system?
- (3) Can we design an MC-side mitigation that provides strong security (no vulnerability windows), avoids the CAM complexity of Sigries, and meets the performance of Sigries?
- (4) Can such a dual mode scheme be made fully deterministic, thus avoiding the reliance on any probabilistic fall-back?

III. ANALYZING SECURITY OF SIGRIES

The two modes of Sigries are designed to be secure individually. The Sigries paper mentions that the transition from heavy-mode to lite-mode is *always* safe. In this section, we show a new vulnerability that targets the other transition. Furthermore, we also analyze the impact of the default vulnerability target of Sigries on the system Mean-Time-To-Failure with an attack that targets multiple sub-banks.

A. Attacking the Second Transition

The budget of lite-mode. On entering lite-mode, the TinyMG tracker is reset. A row can therefore accrue at most T_{MG} activations before the tracker forces a mitigation. As Sigries does not disclose T_{MG} , we credit it with $T_{MG} = T_{RHD}/2$, to securely handle the reset of the tracker against an attacker who straddles a reset with T_{MG} activations on either side. Lite-mode is thus secure *exactly* at its design point, with no slack, and only if the row enters with zero unmitigated activations.

The budget of heavy-mode. PARA is configured with $p = 20/T_{RHD}$. This probability is chosen so that a row is very unlikely to reach T_{RHD} activations without a mitigation. Critically, PARA is provisioned against a budget of T_{RHD} , not T_{MG} : by its own design point, a row in heavy-mode is permitted to accumulate up to T_{RHD} unmitigated activations, which is $2\times$ the budget that lite-mode can absorb.

The composition is unsafe. Let C denote the unmitigated activations a row carries out of heavy-mode. The total activations the row can accrue between two refreshes of the victim is $C + T_{MG}$. Security requires $C + T_{MG} \leq T_{RHD}$, i.e. $C \leq T_{MG}$. But nothing in heavy-mode enforces this: PARA is calibrated to bound C at T_{RHD} , which is twice the permissible value for a safe entry into the lite-mode. At the design point of PARA, the total reaches $1.5 \times T_{RHD}$. Sigries deems this transition safe because it reasons about lite-mode in isolation, assuming lite-mode starts with zero unmitigated activations, however, the guarantee is voided by what the preceding mode hands it.

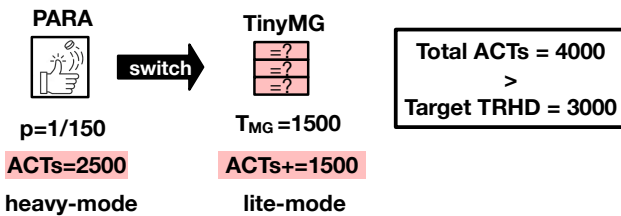


Fig. 3. Example attack: In heavy-mode, PARA permits 2500 unmitigated activations (within T_{RHD} of 3000), then the lite-mode permits an additional 1500 activations, for a total of 4000 activations, which violates security.

The attack. Figure 3 shows our attack for the default T_{RHD} of 3K. PARA permits 2500 unmitigated activations on the attacked row in heavy-mode, within the target of 3000. On switching back to lite-mode, the row accrues an additional $T_{MG} = 1500$ activations, for a total of 4000 across the two modes without a single mitigation, which violates security.

B. Understanding the Impact of the New Vulnerability

As p is calibrated with an escape probability of e^{-20} per aggressor, the target victim failure probability under double-sided attack is e^{-40} . Under our attack, each aggressor carries a T_{MG} budget across the transition, so each aggressor fails with a probability of $(1 - p)^{(T_{RHD} - T_{MG})} = e^{-10}$, and the victim will fail with probability e^{-20} , which is $e^{20} = 4.9 \times 10^8$ higher than the steady-state failure rate of heavy-mode.

As the vulnerability exists at both transitions, an epoch of E windows contains two vulnerable windows rather than one, doubling the exposure of Sigries to two hours per year per sub-bank. More importantly, a budget expressed as a few hours per year *per sub-bank* is an inadequate way to assess system security, as it bounds how long one sub-bank is exposed and says nothing about the failure rate of a system with thousands of them. The next section shows that such a target results in an unacceptably low Mean-Time-To-Failure (MTTF) under a *Round-Robin Attack* that attacks all sub-banks in turn.

C. Round-Robin Attack: A Window is Open Most of the Time

The default Epoch size (without continuous attack on the sub-bank) is 8760. Ideally, if there were 8760 sub-banks in the system, we could devise a pattern that goes through all the sub-banks in a *Round-Robin* fashion, attacking only one sub-bank at a time, as shown in Figure 4. We observe that Cobalt 200 has 12 memory channels [1], each DDR5 channel has 64 banks, and with 8 sub-banks per bank, we have $12 \times 64 \times 8 = 6144$ sub-banks, slightly less than the 8760 we need.

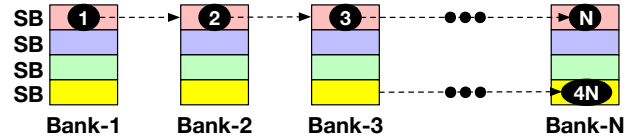


Fig. 4. Round-Robin Attack on a system with N banks, and 4 sub-banks per bank. It attacks only one sub-bank per t_{REFW} , goes round-robin across all sub-banks, and goes through all the sub-banks in $4N$ t_{REFW} .

Our *Round-Robin Attack* targets the 6144 sub-banks, staggered by one t_{REFW} each, so in any t_{REFW} only one sub-bank is under attack. It then waits the remaining $8760 - 6144 = 2616$ windows for the first sub-bank to fall back to lite-mode, and repeats. A vulnerability window is thus open $6144/8760 = 70\%$ of the time, rather than the $1/8760$ that the per-sub-bank budget suggests. To attack a sub-bank within t_{REFW} we perform 4K activations to each of the 33 rows that map to the same sub-bank in a circular fashion.

Evading the Attack Detector: Sigries extends the epoch when its shadow counters indicate that an attack is still ongoing. Our pattern bypasses this detector. A sub-bank gets hammered in one window and receives zero activations for next $E - 1$ windows. As only one window per epoch appears anomalous, each sub-bank would return to lite-mode after E windows.²

²The success of our attack does not hinge on the exact Epoch size. Even if every entry into heavy-mode elongated the epoch by $10\times$, the MTTF under our attack would be 10 seconds, still seven orders of magnitude below PARA.

D. Analyzing the MTTF Under Round-Robin Attack

The key challenge in determining the MTTF of Sigries is converting a vulnerability window into the probability that a victim row fails. Assuming the sub-bank is already in the lite-mode, when the attack starts, the total activation to each row gets distributed into three parts: (1) The activations (A_1) that may occur before the reset of the TinyMG, (2) The activations (A_2) that are needed for overflowing the TinyMG tracker, and (3) The leftover budget (B) for PARA to perform sampling-based mitigation. Figure 5 shows such a split.

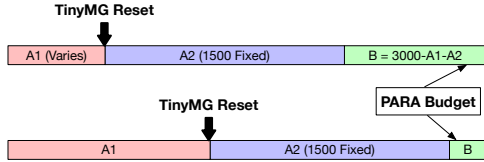


Fig. 5. Activation budget (TRHD) is split into three parts: A_1 (before TinyMG reset), A_2 to overflow TinyMG, and B is the leftover budget for PARA. In windows where A_1 is high, B is severely low for PARA-based sampling.

We pessimistically assume³ that the attack is not synchronized with the reset of the TinyMG tracker (such reset occurs periodically, once per t_{REFW}), so for some of the attack windows, the reset of the tracker will cause a non-zero (A_1) number of activations to be forgotten from the tracker. By design, A_2 equals $T_{MG} = T_{RHD}/2$. Thus, the leftover activation budget for PARA, $B = T_{RHD} - A_1 - A_2$. We note that the activations for A_1 and A_2 escape mitigation by construction. Only B is sampled, so the failure probability of the victim is decided by the budget available to PARA.

The share of A_1 is not fixed: We note that as our attack is not synchronized with the TinyMG reset, A_1 is a dynamic quantity. The tracker reset is periodic in t_{REFW} and is orchestrated by the memory controller, without regard to the access pattern. The attack is therefore not synchronized with the reset, which may land anywhere within it. As the pattern delivers activations at a constant rate, and the tracker forces a mitigation once a row reaches T_{MG} , if a reset happens before the overflow of the TinyMG tracker, the value of A_1 would be uniformly distributed on $[0, T_{MG}]$, and so the budget left to sampling is also uniformly distributed, $B = T_{MG} - [0, T_{MG}]$.

Count chances for PARA, not activations. It is intuitive to think of PARA in terms of the number of chances for mitigation that the budget provides. A budget of B activations buys the defense, on average, $\lambda = p \cdot B$ chances to catch the aggressor, and the aggressor escapes only if all of them miss. At $p = 1/150$, a budget of 1500 provides 10 chances and the likelihood of missing all is e^{-10} , whereas a budget of 150 provides one chance and the likelihood is e^{-1} . Reducing the budget causes exponential decay in the effectiveness of PARA.

³An attacker can learn the periodic schedule of TinyMG reset using side channels, and synchronize [19] the attack such that $T_{MG} - 1$ activations to each aggressor are performed right before the reset of the TinyMG tracker, thereby exhausting almost all the activation budget and leaving almost nothing for PARA. This would increase the probability of failure for each vulnerability window to almost 1. We do not consider such an attack.

TABLE V

ESCAPE PROBABILITY AS THE RESET DONATES MORE OF THE BUDGET. EACH DECILE OCCURS ONE TIME IN TEN; B AND λ ARE AT ITS MIDPOINT.

A_1 (% of T_{MG})	B	$\lambda = pB$	ProbAggRow	ProbVicFail
0–10	1425	9.5	7.3×10^{-5}	5.3×10^{-9}
10–20	1275	8.5	2.0×10^{-4}	3.9×10^{-8}
20–30	1125	7.5	5.4×10^{-4}	2.9×10^{-7}
30–40	975	6.5	1.5×10^{-3}	2.2×10^{-6}
40–50	825	5.5	4.0×10^{-3}	1.6×10^{-5}
50–60	675	4.5	0.011	1.2×10^{-4}
60–70	525	3.5	0.030	8.9×10^{-4}
70–80	375	2.5	0.081	6.6×10^{-3}
80–90	225	1.5	0.222	0.049
90–100	75	0.5	0.606	0.367
Average	—	—	0.096	0.042

Table V varies A_1 across its range in steps of one tenth; as A_1 is uniform, each row occurs one time in ten, so their average is the escape probability the attacker faces. The last two columns give the resulting failure probabilities: $\text{ProbAggRow} = (1-p)^B \approx e^{-\lambda}$ is the chance one aggressor escapes all λ chances, and, as a double-sided attack needs *both* aggressors to escape, $\text{ProbVicFail} = \text{ProbAggRow}^2$. For the first five rows the escape probability is negligible; the risk is concentrated in the last two, where fewer than one expected mitigation remains and the sampler almost always misses. The final row alone contributes 0.061 of the 0.096 average. As each vulnerability window draws a different A_1 , averaging ProbVicFail over its uniform distribution gives a victim-row failure probability of 0.042 per vulnerability window.

The Resulting MTTF. If a victim fails in a vulnerability window with 4.2% probability, then we need 24 vulnerability windows on average to cause a failure. Given 70% of the windows are vulnerable, we need $24/0.7 = 34$ windows. As each t_{REFW} window is 32 milliseconds, **the Mean-Time-to-Failure is about 1 second**. For comparison, we configured PARA with p based on *one failure per 10K years per bank*, which for Cobalt 200 means a system MTTF of 13 years under continuous attack on all banks.⁴ Thus, relative to PARA, Sigries **lowers the MTTF by eight orders of magnitude**.

E. Takeaway: Duty Cycle Bounds Exposure, Not Failure

The security of Sigries rests on $E \geq 8760$, which bounds *how often* a vulnerability window is open but says nothing about the probability of failure *within* it, which we show to be 4.2% rather than the e^{-40} that heavy-mode targets. Once the Round-Robin Attack raises the fraction of vulnerable windows to 70%, the in-window failure probability results in an unacceptably low guarantee in practice.

Ideally, we want an MC-side mitigation that avoids both the vulnerability windows of Sigries and the CAM complexity of Sigries. The next section develops such a solution.

⁴The comparison understates the gap: the 13-year MTTF for PARA assumes all 768 banks are activated continuously at maximum rate (100% of system activation budget), whereas our attack touches one bank per t_{REFW} at one-fifth of its maximum rate, consuming under 0.03% of system activation budget.

IV. FiRM: FILTERED ROWHAMMER MITIGATION

In this section, we design a fully secure and complexity-effective Rowhammer mitigation. Our proposal, *FiRM (Filtered Rowhammer Mitigation)*, is based on two key insights:

Insight-1: The two modes share one budget. For security, the quantity that must be bounded is the number of activations a row receives between two mitigations, and this budget is consumed by whichever mode happens to be active. Sigries allocates the full budget T_{RHD} to each mode separately, so a row that visits both can spend it twice. FiRM instead *partitions* T_{RHD} across the *filter* and the *fallback*, so that the sum over any mode sequence remains within the threshold.

Insight-2: The filter needs only one bit of information. The TinyMG of Sigries is designed to identify *which* row is hot, which is why it needs tags and an associative search. But once the fallback carries the security guarantee, the first stage only needs to determine if the fallback should be used or not. FiRM therefore replaces the CAM-based filter of Sigries with an untagged, direct-mapped table of counters that tracks if the entry has accumulated a given number of activations. Crucially, aliasing in such a table is *safe*: several rows sharing a counter cause it to over-count, so the counter is an upper bound on the activations of any row that maps to it. Over-counting can cost performance, but does not cost security. We note that, even for Sigries, the job of mode-switching is mainly regulated by the spill-counter (with c entries, the mode can switch after $(c + 1) \cdot T_{MG}$ activations to the sub-bank).

We develop two variants of FiRM based on the type of fallback algorithm: *FiRM-P (probabilistic)* and *FiRM-D (deterministic)*. In this section, we focus on FiRM-P and the next section discusses FiRM-D.

A. FiRM-P: Design and Operation

Figure 6 shows an overview of FiRM-P. FiRM uses an SRAM-based filter containing F entries. This is equivalent to a bank containing F sub-banks, each with one filter entry. The filter entry is simply an activation counter that is incremented on activation to the region. Thus, our filter avoids tags and the complexity of CAM. If the value of the accessed counter is below a *Filtering Threshold* (T_F), no mitigation is needed. This is the *lite-mode*. However, if the count exceeds T_F , FiRM-P enters *heavy-mode* where probabilistic mitigation is employed. The design remains in *heavy-mode* for the rest of that window and for $E + 2$ full windows thereafter. Note that the value of E determines the asymptotic performance under attacks and does not impact security.

Filtering Threshold. T_F is bounded from above by lite-mode security and from below by benign behavior. As the filter is cleared every t_{REFW} and the refresh may not be aligned with filter-reset, an attacker can accrue T_F activations on either side of the reset, so lite-mode alone requires $2 \cdot T_F \leq T_{RHD}$. From below, T_F must exceed the largest counter value any benign workload produces, otherwise the workload trips into heavy-mode and pays mitigations it does not need. For our default

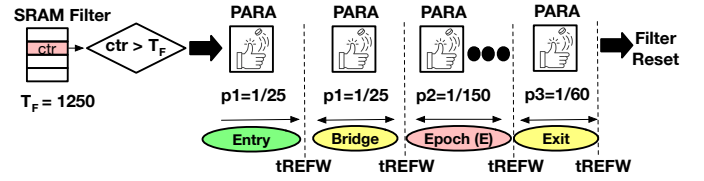


Fig. 6. Overview of FiRM-P. FiRM-P splits the activation budget (3K) between the filtering threshold (T_F) and PARA. On crossing T_F it samples at $p_1=1/25$ for the rest of the *entry* window and for one further *bridge* window (with the filter disabled), then spends E windows at $p_2=1/150$, and one *exit* window at $p_3=1/60$ (based on T_F) before resetting the filter and reusing it.

T_{RHD} of 3K, we use $T_F = 1250$, which satisfies the security bound of 1500 with a margin of 500 activations left for the heavy-mode within the given t_{REFW} window.

Non-uniform sampling. Unlike Sigries, FiRM-P does not use a single p throughout heavy-mode. It instead relies on three distinct probabilities: p_1 (entry into heavy-mode), p_2 (steady-state in heavy-mode), and p_3 (exit to lite-mode). A row can enter heavy-mode carrying up to $2 \cdot T_F$ activations accrued because of the filter (doubled due to filter reset), so the budget available to sampling is reduced within that t_{REFW} window. So, FiRM-P uses a reduced value of p_1 (e.g., $p_1=1/25$) until the t_{REFW} window finishes. As the filter can cross T_F in the middle of a window, a row can carry up to T_F activations into the *next* window as well, so FiRM-P spends one more full window, which we call the *bridge* window, at p_1 with the filter disabled ($T_F=0$). In subsequent steady-state (E) windows, PARA has the full activation budget, so FiRM-P uses a low value of p_2 (e.g., $p_2=1/150$). Finally, to ensure that the heavy-mode to lite-mode transition remains safe, there is one additional *exit* window executed with probability p_3 , which is determined by T_F instead of T_{RHD} (e.g. $p_3=1/60$).

B. Impact of Epoch on Effective Probability of Heavy-Mode

In heavy-mode, Sigries samples at $p = 1/150$ (for T_{RHD} of 3K), and this rate determines its slowdown. FiRM-P samples at $6 \times$ this rate during the entry and bridge windows, so the slowdown in those windows is correspondingly higher. However, a transition window is followed by a long run of steady-state windows sampled at p_2 , which is identical to the rate used by Sigries. The elevated rate is therefore amortized, and the average mitigation rate of FiRM-P is similar to Sigries.

Table VI shows the effective probability of both designs as E is varied from 100 to 10K. At $E = 100$ FiRM-P issues 11% more mitigations than Sigries, but the gap closes to within 1% by $E = 1000$. At the operating point of Sigries ($E \geq 8760$), the two designs are indistinguishable in mitigation rate.

TABLE VI
AVERAGE MITIGATION PROBABILITY OF FiRM-P AND SIGRIES.

Epoch (E)	FiRM-P (p)	Sigries (p)	Ratio
100	1/135	1/150	1.11×
1000	1/148	1/150	1.01×
10000	1/150	1/150	1.00×

The role of E in the two designs is fundamentally different. For Sigries, E is a *security* parameter, which determines vulnerability. For FiRM-P, E is purely a *performance* parameter. Security is established within every window, in both modes and across both transitions, by the partition of the activation budget between the filter and the fallback. The next section establishes the security guarantee formally.

C. Security Analysis

We must show that no row receives more than T_{RHD} activations between two refreshes of the victim. Two facts simplify the argument. First, a refresh interval spans at most one reset and at most one mode boundary. A row that sits just below T_F when the filter is cleared starts again from zero, so a single refresh interval can be charged *two* filter allowances, one on each side of the clear. Second, a fresh allowance of T_F is granted only by a lite-mode window; the bridge and heavy-mode windows run with $T_F = 0$. An interval in which the filter contributes A activations thus leaves $T_{RHD} - A$ to the fallback, and is secure if the rate in force satisfies $p \geq 20/(T_{RHD} - A)$. We apply this test to each mode and boundary of Figure 6.

Case 1: Lite-mode. A counter is an upper bound on the activations of every row that maps to it (aliasing only makes it over-count), so a row must be within T_F activations after the filter reset to stay in lite-mode. A refresh interval that straddles a reset admits T_F on either side, so $A = 2 \cdot T_F = 2500$, which is within T_{RHD} . Security in lite-mode is thus *deterministic* and does not rely on fallback.

Case 2: Heavy-mode. Filtering is disabled, so $A = 0$ and the full budget is available to sampling; the rate may be as low as $20/T_{RHD} = 1/150$, which is the steady-state rate p_2 , which is sufficient to ensure security. The bridge and exit windows sample faster, and the boundaries between heavy-mode windows charge no allowance, so this case covers them.

Case 3: Entry (Lite-mode to Bridge). The refresh interval containing the crossing has a lite-mode prefix that may straddle a filter reset, so $A = 2 \cdot T_F$, leaving only $T_{RHD} - 2 \cdot T_F = 500$ activations for the suffix. Sampling at $p_1 = 20/(T_{RHD} - 2 \cdot T_F) = 1/25$ bounds the suffix to this reduced budget, and provides a failure rate that is equal to the target of PARA. The filter and the fallback are thus charged 2500 and 500 activations, which sum to exactly T_{RHD} .

Case 4: Bridge to Steady-State. The entry window alone is not sufficient to ensure security of steady-state heavy-mode, as the filter crossing can occur at any point within a window. If it occurs near the end, the row carries its T_F filter allowance into the *next* window having paid almost no sampling for it. The interval straddling that boundary is therefore charged $A = T_F = 1250$ with no new allowance in its suffix, which requires $p \geq 20/1750 = 1/87$. The bridge window supplies $p_1 = 1/25$ and is therefore secure. Relaxing to p_2 at this boundary would not be secure: the suffix would fail with probability $(1 - p_2)^{1750} = e^{-11.7}$, more than three orders of magnitude above the target. This is why the entry spends one full window at p_1 with $T_F = 0$ before relaxing to p_2 .

Case 5: Exit (Heavy-Mode to Lite-Mode). The filter is reset on exit, so the lite-mode suffix of the straddling interval contains no further reset and contributes at most $A = T_F$. The heavy-mode prefix must therefore be bounded by T_F as well, which is what $p_3 = 1/60 \geq 20/T_F$ provides, since $(1 - p_3)^{T_F} = e^{-21} < e^{-20}$. The exit is thus the mirror of Case 1, with the carry-out of heavy-mode playing the role of the first T_F and the lite-mode suffix the second, for the same total of $2 \cdot T_F$. As the epoch counter is known to the controller, the exit window is identified in advance and p_3 applied to it.

No vulnerability windows. In every case the activation budget is partitioned so that the filter and the fallback together stay within T_{RHD} , and the resulting failure probability never exceeds the target rate of e^{-20} . FiRM-P therefore has no window in which the guarantee is weaker than in steady state, and needs no vulnerability duty-cycle argument to bound its exposure. Consequently, the multi-sub-bank attack of Sigries does not apply: there is no window for an attacker to schedule. This also explains why E is free of security implications, as it does not appear in any of the five cases above.

D. Storage and Complexity Analysis

Storage. A FiRM filter entry is a bare counter, requiring $\lceil \log_2 T_F \rceil = 11$ bits. A TinyMG entry adds a 14-bit tag (row address within a sub-bank) and valid and lock bits to an identically sized counter, so only 40% of its 27 bits holds the information the mode-switch actually consumes. Table VII compares the two at equal entry count: FiRM needs 352 bytes per bank⁵ against 864 for Sigries, a reduction by $2.5\times$, entirely from discarding metadata rather than tracking fewer regions.

TABLE VII
STORAGE COST AND LOOKUP COMPLEXITY PER BANK (T_{RHD} OF 3K).

	Sigries	FiRM-P
Bits per Entry	27 (14 tag + 2 + 11)	11
Entries per Bank	256	256
Storage per Bank	864 B	352 B (0.4x)
Lookup	32-way CAM	Direct-Mapped

Complexity. The gap in structure matters more than the gap in bits. Every activation to a Sigries sub-bank drives a 32-way associative match, and on a miss a second 32-way reduction to locate the minimum-count entry for eviction. FiRM eliminates the structure rather than shrinking it: an activation indexes the array, performs one SRAM read-modify-write, and compares against T_F . No tags, no associative match, no eviction policy.

⁵FiRM-P also needs to count the number of windows during the heavy-mode. We avoid dedicated storage for this by repurposing the filter-counter in heavy-mode to count windows. If the counter is at most 1250, it represents lite-mode, and activations increment the counter. If the counter is 1251, it represents the *entry-window*, and if it is 1252, it represents the *bridge-window* (PARA with p_1 is used for both), and the counter is incremented only *once* per tREFW. If the counter is between 1253 and 2046, it represents steady-state heavy-mode, so PARA with p_2 is used. If the counter reaches maximum (2047), it represents the *exit-window* (PARA with p_3 is used). When the counter rolls over to zero, it represents lite-mode. Thus, the default Epoch (E) for our FiRM-P implementation is $2046 - 1253 + 1 = 794$ windows.

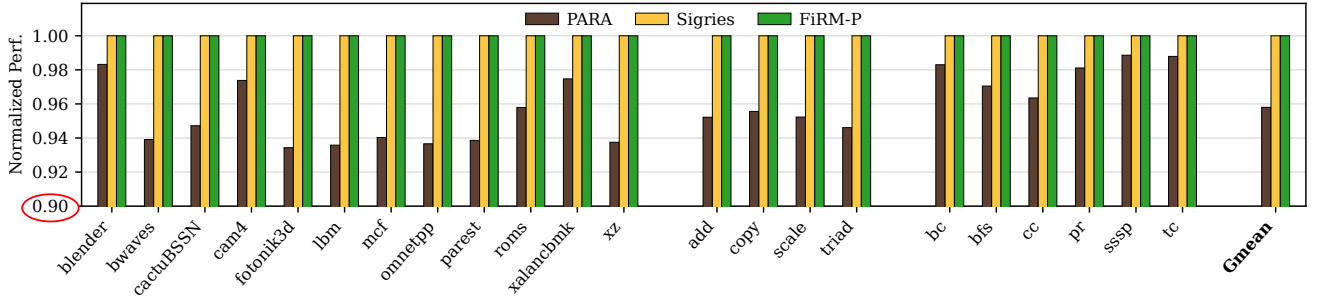


Fig. 7. Performance of PARA, Sigries, and FiRM-P. PARA incurs an average slowdown of 4.4%, whereas Sigries and FiRM-P have no slowdown.

E. Performance Impact for Benign Workloads

1) *Simulation Framework*: We use DRAMSim3 [15], a detailed memory system simulator, to model the DDR5 configuration. Table VIII shows the configuration for our system.

TABLE VIII
BASELINE SYSTEM CONFIGURATION

Out-of-Order Cores	8 cores at 4GHz, 4-wide, ROB-256
Last Level Cache (Shared)	16MB, 16-Way, 64B lines, LRU
Memory bus speed	3 GHz (6000 MT/s)
Channels x Sub-Channels	1 (one 32GB DIMM)×2
Banks x Ranks x Rows	32×1×128K
Page Closure	Adaptive Close Page
Address Mapping	Minimalist Open Page (MOP) [10]

2) *Workload Characterization*: We use twelve high-MPKI benchmarks from SPEC2017 [35], six from GAP [30] and four from STREAM [18]. We run applications in 8-core rate mode for 250 million instructions each. We use weighted speedup as our metric. Table IX shows our workload characteristics. The average activations per bank is 14.8 per-tREFI, so 121K per tREFW. Given 256 entries, each filter entry of FiRM-P receives an average of 474 activations per tREFW, and the most active entry we observe stays well below our T_F of 1250.

TABLE IX
WORKLOAD CHARACTERISTICS.

Workload	MPKI	Avg. ACTs/Bank (per tREFI)	Max ACT/Row (per tREFW)
bc	58.8	13	127
bfs	30.9	12.8	19
cc	57.9	18.4	63
pr	57.7	12.9	19
sssp	27.2	11.9	28
tc	87.8	12.4	20
blender	1.1	5	28
bwaves	41.6	17.1	39
cactuBSSN	3.5	14.6	50
cam4	3.7	9.7	36
fotonik3d	26.6	20.3	75
lbm	27.7	20.3	75
mcf	18.9	19.9	101
omnetpp	9.2	17.4	139
parest	26.6	17.4	312
roms	7.9	12.6	129
xalancbmk	1.6	8.2	39
xz	5.2	17.7	153
add	62.5	16.7	15
copy	50	16.1	18
scale	41.7	15.9	15
triad	53.6	16.8	17
Average	31.9	14.9	69

3) *Results*: Figure 7 shows the performance of PARA, Sigries, and FiRM-P, normalized to the baseline with no mitigation. All designs use DRFMs (with DREAM). PARA incurs an average slowdown of 4.4%, violating requirement R1. Both Sigries and FiRM-P always remain in lite-mode, so they incur 0% slowdown for all workloads.

F. Slowdown Under Attacks

We now analyze the behavior of Sigries and FiRM-P under a Rowhammer attack that targets all banks. The attacker can easily bypass the filtering of both Sigries and FiRM-P, so this reduces to their performance in heavy-mode. Figure 8 shows the performance of Sigries and FiRM-P under attacks. Both schemes have similar behavior as they both degenerate into PARA (with $p = 1/150$). The average slowdown is 4.3% and the worst-case slowdown is 7.7%. We note that our slowdown for Sigries under attack is lower than reported in their paper, because Sigries paper used Naive-DRFM and we use DREAM.

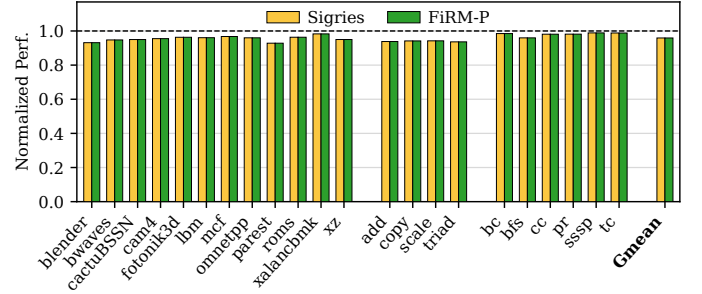


Fig. 8. Performance impact of mitigation of Sigries and FiRM-P under attacks. Both Sigries and FiRM-P have similar slowdowns under attack.

G. Configurability to Rowhammer Threshold

While we design FiRM-P for T_{RHD} of 3K, we can use the same hardware and configure the parameters to target different T_{RHD} . Table X shows the parameters for FiRM-P and slowdown for T_{RHD} of 2K/3K/4K. When FiRM-P is operated at lower than designed T_{RHD} , it can incur slowdown (e.g. 0.5% at $T_{RHD}=2K$). Thus, FiRM-P is configurable.

TABLE X
FiRM-P PARAMETERS AND SLOWDOWN FOR DIFFERENT T_{RHD} .

T_{RHD}	T_F	p_1	p_2	p_3	Avg. Slowdown
2000	750	1/25	1/100	1/37	0.5%
3000	1250	1/25	1/150	1/60	0%
4000	1500	1/50	1/200	1/75	0%

V. FiRM-D: DETERMINISTIC FILTERED MITIGATION

Thus far, all the fallback options we have considered (for both Sigries and FiRM-P) have been probabilistic. Such fallback bounds the overall design to have (a) probabilistic security and (b) reliance on a *Random Number Generator (RNG)*, which can be a source of both complexity and vulnerability [25]. In this section, we show that a filtered Rowhammer mitigation can be built to be fully deterministic and at less than two-thirds the storage overhead of Sigries (and without the CAM complexity). The key idea behind our solution, *FiRM-D (Deterministic)*, is to maintain a counter over a large number of rows and modulate the mitigations such that all rows are refreshed within T_{RHD} activations to the region. The mitigation rate need not be uniform, and can be modulated to be zero below a filtering threshold and higher thereafter.

A. The FiRM-D Substrate: Shared Counters Across Banks

The enabler for the deterministic mitigation is the all-bank variant of DRFM, called $DRFM_{ab}$, which mitigates one sampled row in *every* bank of the sub-channel. A design that spreads the rows sharing a counter *across* banks rather than within one bank can thus mitigate 32 of them per $DRFM_{ab}$ [38]. Figure 9 shows the overview of such a design. Each activation counter (ACTR) is shared by a *gang* containing V rows from every bank (Figure 9 shows $V=2$). Given our budget of 256 counters per bank, we have $256 \times 32 = 8K$ counters across all 32 banks. This means, for our design, V must be set to 16, given each bank has 128K rows. An activation to any row in the gang increments the ACTR of the gang. The mitigation of the gang can be accomplished by sending V $DRFM_{ab}$ commands (with appropriate rows sampled for each bank). This can be done either in bulk or gradually (once per few activations). To support the gradual schemes, each gang also has a *Rotation Pointer (PTR)*, which identifies the row within the gang to be refreshed next. At every tREFW, the ACTR is reset (the mitigation rate must account for loss due to reset), however, the PTR is preserved (to avoid starvation of the last row within the gang).⁶

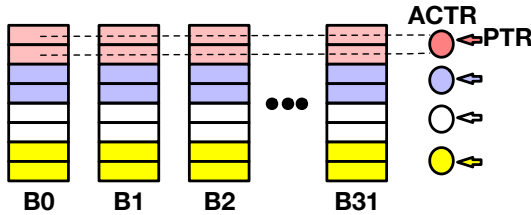


Fig. 9. Substrate for FiRM-D: All the banks share the same activation counter (ACTR). To reduce storage, multiple rows from a bank share the ACTR. For mitigation, $DRFM_{ab}$ is used that mitigates one row from all 32 banks. For V rows per bank, we need V $DRFM_{ab}$ for mitigation. A Rotation Pointer (PTR) per gang identifies which row within the gang to mitigate next.

⁶While Figure 9 shows that the same bank offset from all the banks maps to one ACTR, this is inefficient if a hot page is spread across 32 banks at the same offset, leading to frequent overflow of ACTR. To avoid this, we use rows with different offset in each bank to occupy the same gang, using a per-bank xor mask, similar to the scheme used in DREAM-C [38].

B. FiRM-D Modulation Knob: The Rate of Mitigation

Our main control knob is how the V mitigations are performed for the gang. We consider the three options:

DREAM-C (Bursty Mitigations): DREAM-C is trigger-based. When the counter reaches $T_{RHD}/2$, the whole gang is mitigated at once with V back-to-back $DRFM_{ab}$. Unfortunately, this can stall the channel for several microseconds ($6.6 \mu s$ with $V=16$). The microsecond-scale delay violates the liveness requirement (R3). The advantage of DREAM-C is that it has zero mitigations if the ACTR remains below $T_{RHD}/2$. We note that, as DREAM-C performs the mitigation of the entire gang in bulk, it does not need the per-gang PTR.

Gradual (Paced Mitigation): The burstiness of mitigation can be amortized by pacing: issue one round every X activations to a gang and advance the rotation pointer (PTR), so successive rounds cover successive rows within the gang. Pacing also cuts the reset tax to at most X instead of $T_{RHD}/2$. Thus, one $DRFM_{ab}$ must be sent every $X = T_{RHD}/(V+1) = 176$ activations. At every tREFW, while we reset the ACTR, we must preserve the rotation pointer (PTR), otherwise, the last row in the gang will get starved, leading to security violations. The problem with the Gradual mitigation is that the paced mitigation rate applies at *all* times, even if the application is not under an attack, incurring an average slowdown of 38%.

FiRM-D (Threshold-Based Mitigation): Similar to FiRM-P, FiRM-D splits the activation budget into two parts: the first is a filtering threshold T_F , below which no mitigation is applied, and the remaining budget ($T_{RHD} - 2 \cdot T_F$) is split across the V mitigations. The factor of $2 \cdot T_F$ exists because the filter is reset every tREFW, potentially forgetting up to T_F activations. Similar to the gradual scheme, at every tREFW, while we reset the ACTR, we must preserve the rotation pointer (PTR), otherwise it can lead to security violations for the last row.

Figure 10 shows an overview of the three schemes, for a gang containing four rows from each bank. DREAM-C avoids mitigations until $T_{RHD}/2$ but then causes a burst of four mitigations in bulk. The Gradual scheme pays constant overheads even when the system is not under an attack, and this may be unacceptably high for benign workloads. FiRM-D balances both: it avoids mitigations until T_F , then performs paced mitigation, albeit at a faster rate than the Gradual design.

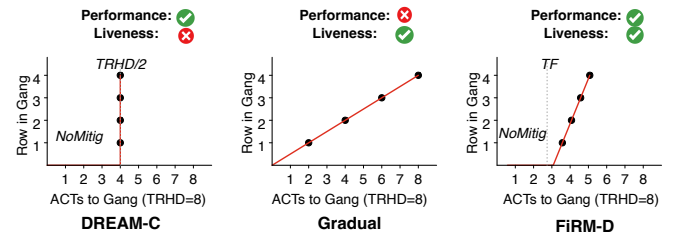


Fig. 10. Three schemes for a gang with $V=4$ and T_{RHD} of 8. DREAM-C is bursty, Gradual has mitigations even for benign workloads, FiRM-D has filtering (no mitigations until T_F and paced mitigations thereafter).

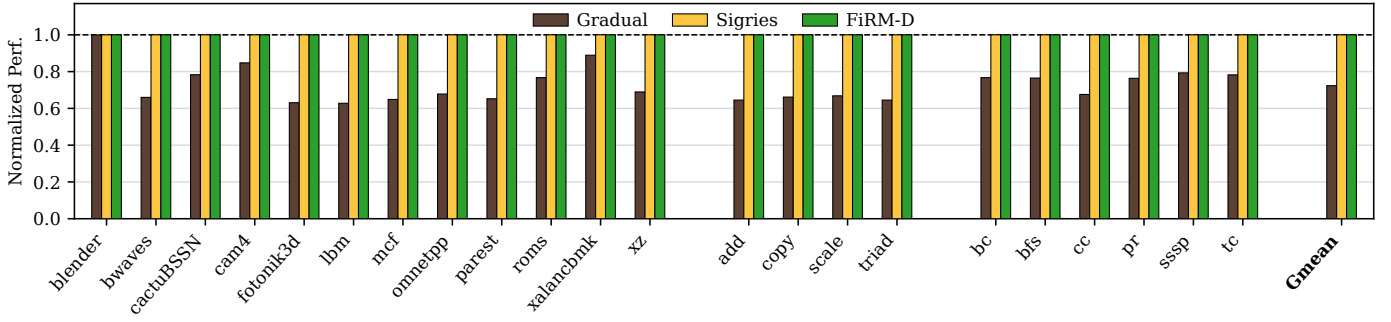


Fig. 11. Performance of Gradual, Sigries, and FiRM-D. Gradual incurs an average slowdown of 38%, whereas Sigries and FiRM-D have zero slowdown.

C. Parameters

All three schemes can be described with two parameters: T_F and APD (ACTs per DRFM). Table XI shows the parameters for the three schemes for our target T_{RHD} of 3K. We assume a gang size containing $V=16$ rows from each bank.

TABLE XI
DETERMINISTIC MITIGATION OPTIONS AT T_{RHD} OF 3K.

Scheme	Gang Size (V)	Filter Th. (T_F)	ACTs per DRFM (APD)	Stall per Mitig. (ns)
DREAM-C	16	1500	0 (for 16)	6600
Gradual	16	0	176	411
FiRM-D	16	1250	29	411

D. Bounding Performance Under Attack

We note that, once the filtering threshold is breached, the mitigation rate of FiRM-D is 6x higher (1-per-29 vs. 1-per-176) than the Gradual scheme. This occurs because the Gradual scheme spreads its 16 mitigations over the entire T_{RHD} , however, FiRM-D must squeeze its 16 mitigations within a much shorter budget of $T_{RHD} - 2 \cdot T_F$. Thus, under attacks, the Gradual scheme would have 6x lower slowdowns than FiRM-D. To achieve the best of both worlds (zero slowdown for benign applications and better bounds under attacks), we take an epoch-based approach, similar to FiRM-P. Figure 12 shows the overview of our FiRM-D-Epoch scheme. It remains in FiRM-D (Mode-00) until the ACTR is below T_F . Then, at the end of tREFW, it moves to a *fast* Gradual scheme (Mode-01) that uses a rate of 1-per-29 ACT for secure transition and lasts for exactly one tREFW. Then, it moves to a Gradual scheme (Mode-10) with a rate of 1-per-176 and lasts for E windows (hundreds of tREFW). Then, before transition, it spends one more tREFW in *fast* Gradual scheme (Mode-11), before finally transitioning back to FiRM-D (Mode-00).

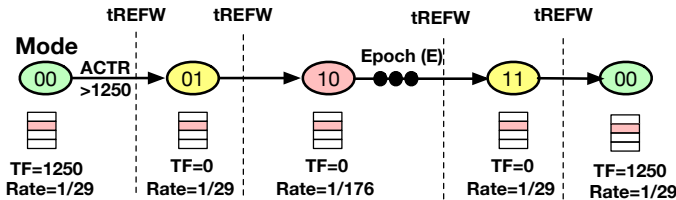


Fig. 12. Epoch-Based FiRM-D (enables better performance under attacks)

E. Security Analysis

Security requires that no row receives more than T_{RHD} activations between two refreshes of the victim. Two facts simplify the argument. First, as the ACTR reset is not aligned with the refresh of a row, a refresh interval spans at most one reset and at most one mode transition. Second, the PTR is preserved across resets and transitions, so a row is served once every V mitigations: at one mitigation per X activations, a row waits at most $V \cdot X$ activations for its turn, plus the up-to $X - 1$ activations of pacing progress discarded at the reset. An interval granting a filtering allowance of A is thus secure if $A + (V+1) \cdot X \leq T_{RHD}$, with X the slowest rate in force.

Case 1: Mode-00 (FiRM-D). An interval straddling a reset receives an allowance of T_F on either side, so $A = 2 \cdot T_F = 2500$ and the rate must satisfy $X \leq \lfloor (T_{RHD} - 2 \cdot T_F) / (V+1) \rfloor = 29$, which is exactly the FiRM-D rate (worst case 2993).

Case 2: Gradual (Mode-01, 10, 11). Filtering is disabled, so $A = 0$ and the entire budget is available to pacing: the rate may be as slow as $T_{RHD} / (V+1) = 176$, which is the Mode-10 rate (worst case 2992). Mode-01 and Mode-11 are faster, and the two transitions Mode-01 to Mode-10 and Mode-10 to Mode-11 also charge no allowance, so this case covers them.

Case 3: Entry (Mode-00 to Mode-01). The transition coincides with a reset, so the interval charges only one allowance, $A = T_F$, leaving room for a rate no slower than $(T_{RHD} - T_F) / (V+1) = 102$. Mode-01, at 1-per-29, is secure; entering Mode-10 directly would not be, as the row could accrue $T_F + 17 \cdot 176 = 4242$ activations.

Case 4: Exit (Mode-11 to Mode-00). This is the mirror of Case 3. At the Mode-10 rate, the last row of the gang could instead wait $15 \cdot 176$ activations for its turn in the exit window and a further $T_F + 29$ after the reset, for a total of 3919. Operating at Mode-11 at a rate of 1-per-29 removes this case.

F. Performance Impact on Benign Workloads

Figure 11 shows the performance of Gradual, FiRM-D, and Sigries normalized to the baseline without any mitigation. For FiRM-D, no workload in our suite drives an entry past T_F (the average ACTR reaches 474 activations, whereas $T_F=1250$), so we expect FiRM-D to issue no mitigations. The Gradual scheme suffers an average slowdown of 38% and up to 60%, whereas both FiRM-D and Sigries have zero slowdown.

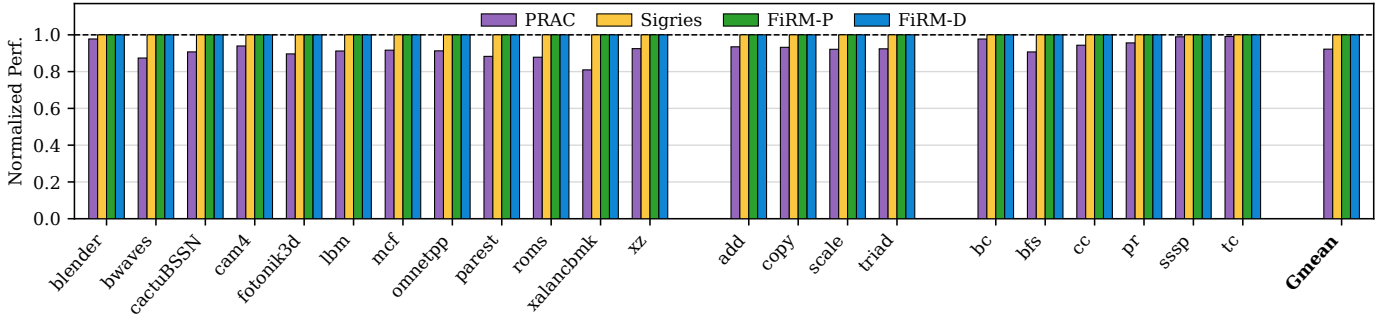


Fig. 13. Performance of PRAC, Sigries, FiRM-P and FiRM-D. PRAC has an average slowdown of 8.4%, whereas the other three designs have zero slowdown.

G. Storage Analysis

Each entry for FiRM-D requires an ACTR (11-bit), a Rotation-Pointer (4-bit), and Mode (2-bit). To reduce storage, we use a single global Epoch counter. Table XII compares the storage of FiRM-P, FiRM-D, and Sigries. FiRM-P requires less than half the storage of Sigries, whereas FiRM-D requires less than two-thirds the storage of Sigries. Both FiRM-P and FiRM-D avoid the CAM complexity of Sigries.

TABLE XII
STORAGE AND COMPLEXITY PER BANK (T_{RHD} OF 3K).

	Sigries	FiRM-P	FiRM-D
Bits per Entry	27 (14+2+11)	11	17 (11+4+2)
Entries per Bank	256	256	256
Storage per Bank	864 B	352 B (0.4 $\times$)	544 B (0.63 $\times$)
Lookup	32-way CAM	Direct-Mapped	Direct-Mapped
Requires RNG	Yes	Yes	No
Security	Probabilistic	Probabilistic	Deterministic

H. Performance Impact Under Attacks

Figure 14 shows the performance of benign workloads when a co-running attacker causes mitigations at the maximum rate. Sigries and FiRM-P incur an average slowdown of 4.3%, whereas FiRM-D incurs 2 $\times$. The gap has two sources: (1) FiRM-D must refresh all the rows in the gang (using V mitigations), whereas FiRM-P does one mitigation to a row. (2) Each mitigation stalls all 32 banks, so benign co-runners absorb stalls even on banks the attacker did not mitigate.

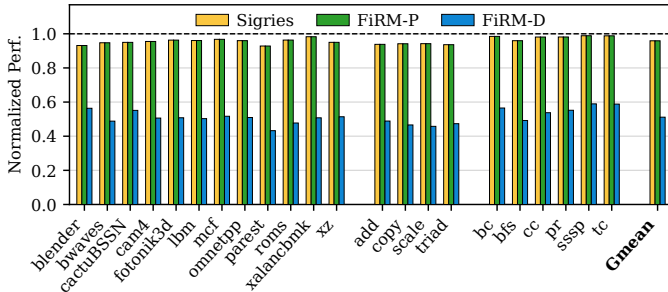


Fig. 14. Performance impact of mitigation of Sigries, FiRM-P and FiRM-D under an attack with maximum mitigation rate.

We argue this cost is acceptable. The rate is bounded and known at design time (at most one $DRFM_{ab}$ per 29 activations), so FiRM-D has no performance tail and meets

R2. The longest stall is a single $DRFM_{ab}$ (411 ns), 16 $\times$ below DREAM-C and well under the microsecond scale that R3 prohibits. The degradation is within the range of what a co-located tenant can already inflict through memory contention alone [22], [24], and it ceases with the attack. FiRM-D thus trades attack-time throughput for deterministic security.

VI. RELATED WORK

PRAC: Recent JEDEC specification introduced *Per-Row Activation Counting (PRAC)*, which maintains an activation count alongside every row [9], [28], [39]. To enable counter update, PRAC increases t_{RC} , so it incurs a slowdown even when the system is not under attack. Figure 13 compares PRAC with Sigries, FiRM-P, and FiRM-D at T_{RHD} of 3K. PRAC suffers 8.4% on average, whereas all three MC-side designs are free on benign workloads. Furthermore, PRAC remains optional for DDR5, whereas the MC-side designs can protect DDR5 DIMMs (that may remain in use over the next decade).

Filtered Mitigation In-DRAM: Our design of FiRM-P is inspired by MIRZA [37], an in-DRAM mitigation that uses filtering and randomization. As MIRZA is in-DRAM, its filter reset is coordinated with refresh, so it does not suffer from security issues in transitioning from one mode to another. As FiRM is MC-based, it has no visibility into when a row is refreshed, which causes both the filter budget to be doubled and security issues in transitioning from randomization to filtering. Finally, FiRM-P is carefully designed to provide a slowdown of 4.3% even under attacks.

VII. CONCLUSION

Sigries is an MC-side Rowhammer defense deployed in production. We show a new vulnerability in Sigries. We also show that by targeting multiple sub-banks in a Round-Robin manner, an attacker can keep a vulnerability window open at almost every instant, reducing the system MTTF by eight orders of magnitude. We propose *FiRM*, a principled filtered mitigation. FiRM partitions the activation budget across the filter and the fallback to ensure security across modes and transitions. FiRM-P uses non-uniform sampling and needs less than half the storage of Sigries. FiRM-D provides deterministic security and needs less than two-thirds the storage of Sigries. Both designs incur zero slowdown for benign workloads. We show that neither the security tradeoff nor the CAM complexity of Sigries is essential for a practical mitigation.

APPENDIX A

ON OPTIMIZING FOR THE BLIND-STATIC ATTACKER

Both Sigries and FiRM assume an attacker who knows the mitigation algorithm and its parameters (Section II-A), therefore, neither design seeks to obtain security through obscurity. Accordingly, we evaluate both the security properties and performance under attack, assuming that the attacker is aware of the algorithm and is trying to cause harm (either failures or slowdown) by crafting the most stressful access pattern based on the knowledge of the underlying defense.

One may argue that the Misra-Gries tracker is more precise than the group-counting for FiRM-P and can provide performance benefits for specifically crafted weaker attack patterns. However, this is a regime which requires the opposite assumption – the attacker is unaware of the defense parameters and is also unwilling to change the attack pattern. The tracker outperforms the filter only on patterns that hammer fewer rows than it has entries, and an attacker who knows the design has no incentive to produce such a pattern. We contend that the precise tracker of Sigries therefore optimizes for a regime that is not meaningful in practice: benign workloads never enter it, and an attacker who may try to launch such patterns has nothing to gain (failures or slowdown) and hence is unlikely to do it for any sustained period of time.

The Regime. Let C denote the number of entries in a TinyMG ($C=32$ in our configuration). A pattern that hammers at most C distinct rows within a sub-bank is tracked exactly: the spill-counter never reaches T_{MG} , Sigries never leaves lite-mode, and a tracked row is mitigated once per $T_{MG}=1500$ activations. FiRM has no notion of *how many* rows are hot, as the filter counter is a per-region activation total, so any pattern that drives a region past T_F trips the fallback. On a 10-row pattern (*decahammer*), Sigries therefore would issue $10\times$ fewer mitigations than FiRM-P (1-in-1500 vs. 1-in-150).

Not for Benign Workloads. Table IX shows that the most active row across our 22 workloads receives 312 activations per t_{REFW} , and the busiest FiRM filter entry stays well below $T_F=1250$. Neither design issues a single mitigation on this suite. Thus, the precise counting of TinyMG is not meant to mitigate the hot rows in benign applications (e.g., the hottest row in the Sigries study [32] had 583 activations, well below the T_{MG} of 1500). So, the differentiator between the tracker and the filter is visible only under deliberate hammering.

Not Attackers With Anything to Gain. An attacker has an incentive to persist only if the attack yields something: a bit-flip, or meaningful slowdown of the system. Decahammer against Sigries delivers neither. It cannot flip a bit, as the tracker forces a mitigation every T_{MG} activations and the row never approaches T_{RHD} . It is also strictly dominated as a denial-of-service vehicle: hammering $C+1$ rows instead of C pushes the sub-bank into heavy-mode and yields 4.3% slowdown rather than 0.4%. Sigries thus wins only where the attacker deliberately settles for $10\times$ lower harm.

A Taxonomy of Attackers. To better understand the landscape of attacks, we classify the attackers along two axes. First, based on the *Knowledge*, the attacker is either *Blind* (the attacker knows nothing of the deployed mitigation) or *Informed* (the attacker knows the algorithm and its parameters). Second, based on *Skill*, the attacker is either *Static* (the attacker runs only a small set of fixed patterns) or *Adaptive* (the attacker varies the pattern and tries to converge on the most effective one). Figure 15 shows the four types of attackers.

		Skill	
		Static	Adaptive
Knowledge	Blind	Advantage <i>(not useful regime)</i>	No advantage <i>(valid regime)</i>
	Informed	No advantage <i>(valid regime)</i>	No advantage <i>(valid regime)</i>

Fig. 15. The four attacker classes. The Misra-Gries tracker of Sigries outperforms the FiRM filter only against a Blind-Static attacker, a regime where the attacker has no incentive to run for meaningful time.

An example of *Blind-Adaptive* is the fuzzing approach of TRRespass [4] and Blacksmith [8], which defeat in-DRAM TRR without any knowledge of its design by searching the space of patterns until one works. *Informed-Static* is the worst-case analysis by which defenses such as ProTRR [17] and MINT [29] are evaluated: the pattern is derived analytically from a known design and applied as given. *Informed-Adaptive* begins from such a pattern and adjusts it to handle runtime contention, for example to exploit *Refresh Postponement* [3]. A *Blind-Static* attacker in theory runs a few canonical patterns, for a short time, whether or not they accomplish anything.

The three classes that describe reported Rowhammer attacks are thus Blind-Adaptive, Informed-Static, and Informed-Adaptive, and in all three cells, Sigries and FiRM behave identically. The tracker earns its advantage only in the one quadrant that provides no incentive to run for a long time.

The Residual Case. What remains is an attacker who is uninformed, unadaptive, and persistent enough to run a failing pattern long enough to matter for system performance, while gaining nothing itself. Even in this case, FiRM-P bounds the attacker to only a small slowdown in the heavy-mode (4.3% on average and 7.7% at worst). A Blind-Static attacker moves FiRM from one point inside the minor slowdown envelope to another. It creates no new worst case, R1 and R2 are unaffected, and the same 4.3% is available against Sigries to any attacker willing to change one constant in the pattern.

Implication. The tracker buys lower slowdown only against attacks that are already failing, and only from an attacker who is simultaneously uninformed and unwilling to experiment. The price is a 32-way associative lookup on every activation, $2.5\times$ the storage, and security vulnerability. Precision is worth paying for when it improves the cases a system encounters for significant time, the Blind-Static quadrant is not one of them.

APPENDIX B ON SETTING THE SAMPLING RATE FOR PARA

The sampling rate p for PARA must be set carefully, as it determines both the security level (the Mean-Time-to-Failure under continuous attack) and the performance overhead (the rate of mitigations). Similar to prior works [7], [27], [29], we target an MTTF of 1 failure per 10K years per bank under continuous attack, as this yields a bank-failure rate comparable to that of naturally occurring errors. For this target, we set p for a given T_{RHD} using the approximation of Equation 1.

$$p = 20/T_{RHD} \quad (1)$$

A more precise rate can be obtained with the recurrence-based method of Saroiu and Wolman [31]. Table XIII compares the two, both as absolute probabilities and as coefficients relative to T_{RHD} . Our approximation tracks the precise method closely, with a coefficient of 20 against 19.02–19.60, and errs on the safe side: it samples 2–5% more often than required, giving PARA slightly more chances to catch an aggressor. The same holds for the fallback of Sigries, so the MTTF we report under the Round-Robin Attack is conservative in this respect as well. The two methods are virtually indistinguishable in performance, as the difference in mitigation rate is negligible.

TABLE XIII
SAMPLING PROBABILITY FROM THE PRECISE SAROIU-WOLMAN COMPUTATION [31] AND FROM THE $20/T_{RHD}$ APPROXIMATION. OUR APPROXIMATION IS 2–5% MORE CONSERVATIVE.

T_{RHD}	Precise		Approximate	
	p	Coefficient	p	Coefficient
500	1/25.5	19.60/ T_{RHD}	1/25	20/ T_{RHD}
1000	1/51.4	19.45/ T_{RHD}	1/50	20/ T_{RHD}
1500	1/77.7	19.31/ T_{RHD}	1/75	20/ T_{RHD}
2000	1/104.3	19.18/ T_{RHD}	1/100	20/ T_{RHD}
2500	1/130.9	19.09/ T_{RHD}	1/125	20/ T_{RHD}
3000	1/157.7	19.02/ T_{RHD}	1/150	20/ T_{RHD}

RESPONSIBLE DISCLOSURE

On July 8, 2026, we contacted the Sigries authors to note that the heavy-mode to lite-mode transition may be unsafe, as the tracker is guaranteed to be secure only if a row enters with zero unmitigated activations, whereas a row leaving sampling-mode may carry some. We shared a draft of this paper with them on August 3, 2026, and have since incorporated their feedback on the description of Sigries. Our analysis of the Round-Robin Attack has been revised since that draft, and the MTTF we report here is lower than the one shared with them. This work analyzes a design as described in a published paper, using estimated parameters. We did not attempt to mount the attack on any deployed system.

IMPACT STATEMENT

Our analysis shows that an attacker running a Round-Robin pattern across sub-banks can bypass the protection Sigries provides, so a system deploying it must rely on in-DRAM mitigations to tolerate Rowhammer. We note that this leaves such a system no worse off than the many servers shipping today with no SoC-level mitigation at all, and that the vulnerability arises from how the two modes are parameterized rather than from any flaw in either mode individually. Our paper shows that our filtered design can retain the benefits of Sigries while remaining secure across modes and transitions.

ACKNOWLEDGMENTS

We thank the Sigries team for describing their design in detail in a research paper. Progress in hardware security depends on avoiding security-by-obscurity, and we commend the Sigries team for opening their design to external analysis. We thank Stefan Saroiu and Sujay Yadalam for discussions at ISCA and for answering our questions.

REFERENCES

- [1] (2025) “Microsoft Rolls Out Cobalt 200 CPU with 132 Arm Cores”. <https://www.techpowerup.com/343062/microsoft-rolls-out-cobalt-200-cpu-with-132-arm-cores>.
- [2] M. V. Beigi, Y. Cao, S. Gurumurthi, C. Recchia, A. Walton, and V. Sridharan, “A systematic study of ddr4 dram faults in the field,” in *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2023.
- [3] F. de Ridder, P. Jattke, and K. Razavi, “Posthammer: Pervasive browser-based rowhammer attacks with postponed refresh commands,” in *34th USENIX Security Symposium (USENIX Security 25)*. Seattle, WA: USENIX Association, Aug. 2025, pp. 5661–5678. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity25/presentation/de-ridder>
- [4] P. Frigo, E. Vannacc, H. Hassan, V. Van Der Veen, O. Mutlu, C. Giuffrida, H. Bos, and K. Razavi, “TRRespass: Exploiting the many sides of target row refresh,” in *2020 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2020, pp. 747–762.
- [5] N. Gholkar, J. Stojkovic, H. Al Maruf, G. Price, P. Chauhan, H. Patel, C. Van Goethem, K. Vemuri, K. Malwankar, K. Sriadibhatla, K. Subramanian, S. Kanaujia, C. Tang, and A. Dhanotia, “Vistara: Making cxl real: Full path from asic design and os support to hyperscale deployment,” in *Proceedings of the 53rd Annual International Symposium on Computer Architecture*, ser. ISCA Industry Track, 2026.
- [6] H. Hassan, Y. C. Tugrul, J. S. Kim, V. Van der Veen, K. Razavi, and O. Mutlu, “Uncovering in-dram rowhammer protection mechanisms: A new methodology, custom rowhammer patterns, and implications,” in *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, 2021, pp. 1198–1213.
- [7] A. Jaleel, G. Saileshwar, S. W. Keckler, and M. Qureshi, “Pride: Achieving secure rowhammer mitigation with low-cost in-dram trackers,” in *ISCA*. IEEE, 2024.
- [8] P. Jattke, V. van der Veen, P. Frigo, S. Gunter, and K. Razavi, “BLACKSMITH: Rowhammering in the Frequency Domain,” in *43rd IEEE Symposium on Security and Privacy (Oakland)*, 2022, https://comsec.ethz.ch/wp-content/files/blacksmith_sp22.pdf.
- [9] JEDEC, “JESD79-5C: DDR5 SDRAM Specifications,” 2024.
- [10] D. Kaseridis, J. Stuecheli, and L. K. John, “Minimalist open-page: A dram page-mode scheduling policy for the many-core era,” in *Proceedings of the 44th Annual IEEE/ACM International Symposium on Microarchitecture*, 2011, pp. 24–35.
- [11] J. S. Kim, M. Patel, A. G. Yağlıkçı, H. Hassan, R. Azizi, L. Orosa, and O. Mutlu, “Revisiting rowhammer: An experimental analysis of modern dram devices and mitigation techniques,” in *ISCA*. IEEE, 2020, pp. 638–651.

- [12] M. J. Kim, S. Baek, J. Kim, H. Nam, N. S. Kim, and J. H. Ahn, "SoK: Systematizing a decade of architectural rowhammer defenses through the lens of streaming algorithms," in *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*, 2026, pp. 1594–1612.
- [13] Y. Kim, R. Daly, J. Kim, C. Fallin, J. H. Lee, D. Lee, C. Wilkerson, K. Lai, and O. Mutlu, "Flipping bits in memory without accessing them: An experimental study of dram disturbance errors," *ISCA*, 2014.
- [14] A. Kwong, D. Genkin, D. Gruss, and Y. Yarom, "Rambleed: Reading bits in memory without accessing them," in *2020 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2020, pp. 695–711.
- [15] S. Li, Z. Yang, D. Reddy, A. Srivastava, and B. L. Jacob, "DRAMsim3: A Cycle-Accurate, Thermal-Capable DRAM Simulator," *IEEE Comput. Archit. Lett.*, vol. 19, no. 2, pp. 110–113, 2020.
- [16] H. Luo, A. Olgun, A. G. Yağlıkçı, Y. C. Tuğrul, S. Rhyner, M. B. Cavlak, J. Lindegger, M. Sadrosadati, and O. Mutlu, "Rowpress: Amplifying read disturbance in modern dram chips," in *Proceedings of the 50th Annual International Symposium on Computer Architecture*, ser. ISCA '23. New York, NY, USA: Association for Computing Machinery, 2023. [Online]. Available: <https://doi.org/10.1145/3579371.3589063>
- [17] M. Marazzi, P. Jattke, F. Solt, and K. Razavi, "Protrr: Principled yet optimal in-dram target row refresh," in *IEEE Symposium on Security and Privacy (SP)*. IEEE, 2022, pp. 735–753.
- [18] J. D. McCalpin *et al.*, "Memory bandwidth and machine balance in current high performance computers," *IEEE computer society technical committee on computer architecture (TCCA) newsletter*, vol. 2, no. 19-25, 1995.
- [19] D. Meyer, P. Jattke, M. Marazzi, S. Qazi, D. Moghimi, and K. Razavi, "Phoenix: Rowhammer attacks on ddr5 with self-correcting synchronization," in *Proceedings of the 2026 IEEE Symposium on Security Privacy (S&P)*, San Francisco, CA, USA, 2026.
- [20] "DDR5 SDRAM Datasheet: Directed Refresh Management (DRFM), Page-290," Micron Technology Inc., 2022. [Online]. Available: https://www.micron.com/-/media/client/global/documents/products/data-sheet/dram/ddr5/ddr5_sdram_core.pdf
- [21] J. Misra and D. Gries, "Finding Repeated Elements," *Science of Computer Programming* 2, pp. 143–152, 1982.
- [22] T. Moscibroda and O. Mutlu, "Memory performance attacks: denial of memory service in multi-core systems," in *Proceedings of 16th USENIX Security Symposium*, ser. SEC'07, 2007.
- [23] Y. Park, W. Kwon, E. Lee, T. J. Ham, J. H. Ahn, and J. W. Lee, "Graphene: Strong yet lightweight row hammer protection," in *MICRO*. IEEE, 2020, pp. 1–13.
- [24] P. Pessl, D. Gruss, C. Maurice, M. Schwarz, and S. Mangard, "Drama: exploiting dram addressing for cross-cpu attacks," ser. SEC'16, 2016.
- [25] S. Qazi and D. Moghimi, "Soothsayer: Bypassing dsac mitigation by predicting counter replacement," *DRAMSec4*, 2024.
- [26] S. Qazi and M. Qureshi, "Drfm and the art of rowhammer sampling," *DRAMSec*, 2025.
- [27] M. Qureshi, "AutoRFM: Scaling Low-Cost In-DRAM Trackers to Ultra-Low Rowhammer Thresholds," in *HPCA*, 2025.
- [28] M. Qureshi and S. Qazi, "MOAT: Securely mitigating rowhammer with per-row activation counters," *ASPLOS-2025*, 2025.
- [29] M. Qureshi, S. Qazi, and A. Jaleel, "Mint: Securely mitigating rowhammer with a minimalist in-dram tracker," in *MICRO*. IEEE, 2024.
- [30] K. A. S. Beamer and D. Patterson, "The gap benchmark suite," in *arXiv preprint arXiv:1508.03619*, 2015.
- [31] S. Saroiu and A. Wolman, "How to configure row-sampling-based rowhammer defenses," in *Workshop on DRAM Security (DRAMSec)*, 2022.
- [32] S. Saroiu, S. Yadalam, A. Wolman, W. Remaklus, D. Berger, I. H. Luna, I. Agarwal, and J. R. Lorch, "From lab to fleet: Building and deploying a practical rowhammer defense in cloud socs," *ISCA (Industry Track)*, 2026.
- [33] A. Saxena, A. Jaleel, and M. Qureshi, "Impress: Securing dram against data-disturbance errors via implicit row-refresh mitigation," in *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2024, pp. 935–948.
- [34] M. Seaborn and T. Dullien, "Exploiting the DRAM rowhammer bug to gain kernel privileges," *Black Hat*, vol. 15, p. 71, 2015.
- [35] "SPEC CPU2017 Benchmark Suite," Standard Performance Evaluation Corporation. [Online]. Available: <http://www.spec.org/cpu2017/>
- [36] L. C. Stefan Saroiu, Alec Wolman, "The price of secrecy: How hiding internal dram topologies hurts rowhammer defenses," in *Proceedings of International Reliability Physics Symposium (IRPS)*, 2022.
- [37] H. Taneja, A. Hajiabadi, M. Marazzi, K. Razavi, and M. Qureshi, "Mirza: Efficiently mitigating rowhammer with randomization and alert," in *IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2026.
- [38] H. Taneja and M. Qureshi, "Dream: Enabling low-overhead rowhammer mitigation via directed refresh management," in *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, 2025, pp. 776–792.
- [39] J. Woo, C. S. Lin, P. J. Nair, A. Jaleel, and G. Saileshwar, "Qprac: Towards secure and practical prac-based rowhammer mitigation using priority queues," *HPCA*, 2025.